

Multi-Agent Orchestrator

Author

Sai Veda

Document date

February 14, 2025

Validation

69 automated tests

Abstract

This project reframes agent systems as workflow infrastructure: planning, scheduling, guardrails, and repair are treated as first-class components instead of being hidden behind one monolithic prompt loop.

Project Background

Most agent demos fail for the same reason: planning quality is inconsistent, execution is brittle, and there is no disciplined repair path when a tool call goes wrong. The brief targets those failure modes directly.

Headline Result

Self-correction lifts task success 41.9% -> 100%

Scope Overview

Planner, scheduler, tool registry, guardrails, and accounting. Deterministic policy engine behind an LLM-like interface. Repair loop based on execution feedback and validator errors. Benchmark suite that compares base execution against self-correction.

Project Context

A from-scratch agentic framework - LangGraph / AutoGPT-class, but a clean, dependency-light design of its own. A planner decomposes a natural-language task into a DAG of tool calls; an executor runs that DAG respecting dependencies and running independent branches in parallel; guardrails validate inputs, tool arguments, and outputs; and a self-correction loop turns real execution feedback into a repaired plan and retries.

Executive Summary

Background, objectives, scope, and project impact.

Objectives

- Decompose tasks into a typed dependency graph.
- Run independent branches safely and in parallel.
- Recover from tool and validation failures through repair loops.

Scope

- Planner, scheduler, tool registry, guardrails, and accounting.
- Deterministic policy engine behind an LLM-like interface.
- Repair loop based on execution feedback and validator errors.
- Benchmark suite that compares base execution against self-correction.

Project Background

Most agent demos fail for the same reason: planning quality is inconsistent, execution is brittle, and there is no disciplined repair path when a tool call goes wrong. The brief targets those failure modes directly.

Executive Summary

This project reframes agent systems as workflow infrastructure: planning, scheduling, guardrails, and repair are treated as first-class components instead of being hidden behind one monolithic prompt loop.

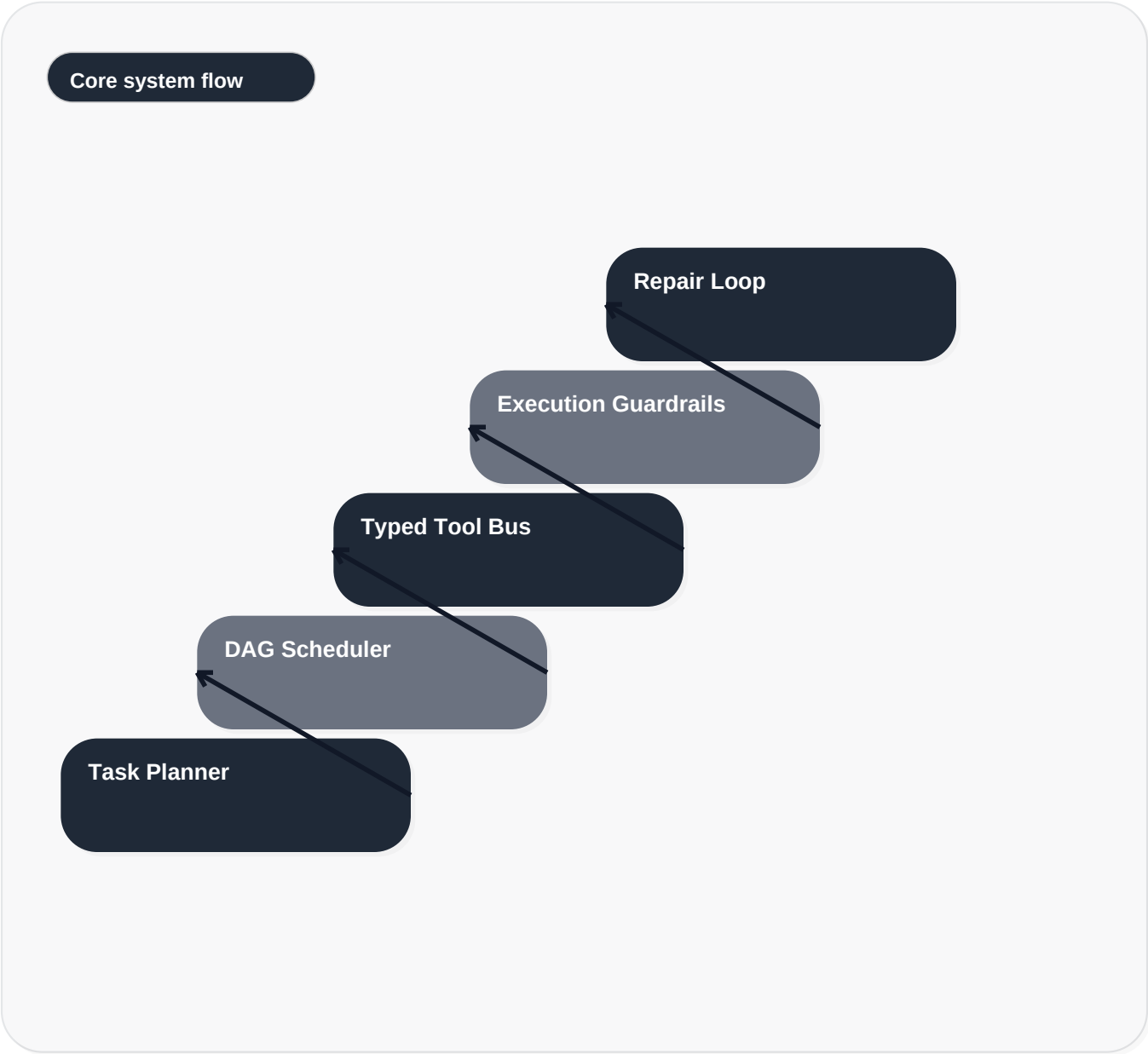
Impact

The self-correction loop is the center of gravity. It raises success from partial completion to full task completion because execution feedback is used to repair plans instead of treating every failure as terminal.

Self-correction lifts task success 41.9% -> 100%

Solution Architecture

The system separates planning, execution, and repair so the orchestrator can recover from tool failures instead of collapsing.



Design Note 1

Task recovery uses execution feedback only, which keeps the success lift defensible.

Design Note 2

Planning and execution are intentionally decoupled so new tools do not rewrite orchestration logic.

Design Note 3

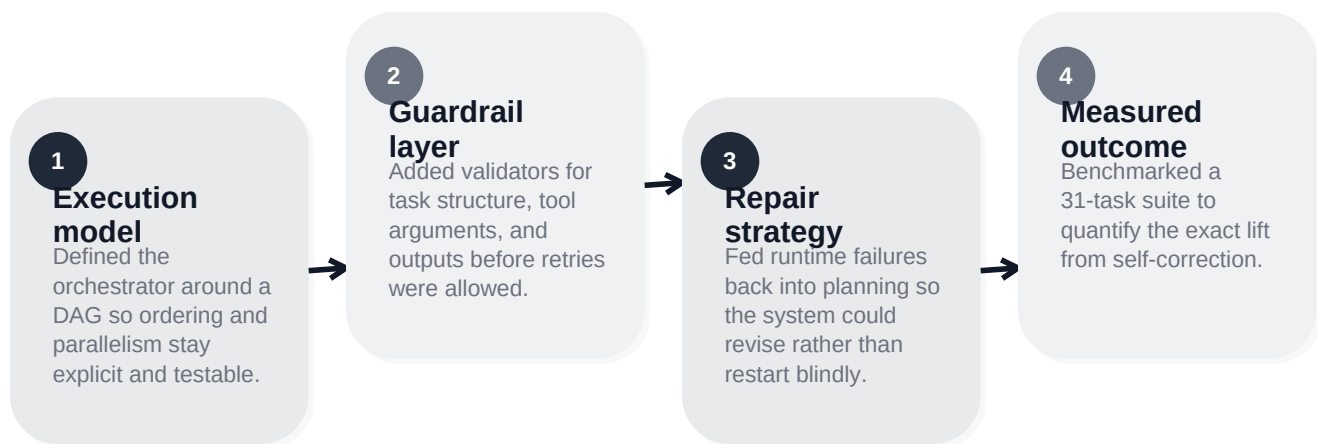
The toolbox and accounting model keep the framework compatible with more capable planners later.

Implementation Process

Delivery phases, engineering approach, and quality checkpoints.

Delivery Narrative

A from-scratch agentic framework - LangGraph / AutoGPT-class, but a clean, dependency-light design of its own. A planner decomposes a natural-language task into a DAG of tool calls; an executor runs that DAG respecting dependencies and running independent branches in parallel; guardrails validate inputs, tool arguments, and outputs; and a self-correction loop turns real execution feedback into a repaired plan and retries.



Quality Checkpoints

- 69 automated tests validate core behavior and edge cases.
- Benchmark files and screenshots are generated from committed project artifacts.
- The run path stays deterministic so numbers can be reproduced and reviewed directly.

Engineering Principles

- Task recovery uses execution feedback only, which keeps the success lift defensible.
- Planning and execution are intentionally decoupled so new tools do not rewrite orchestration logic.
- The toolbox and accounting model keep the framework compatible with more capable planners later.

Evidence And Results

Test evidence, benchmark interpretation, and screenshot review.

69 passing tests

February 14, 2025

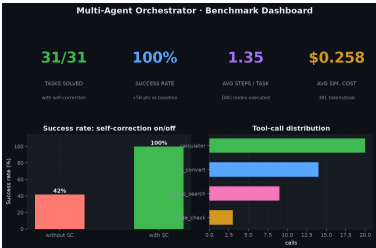
Self-correction lifts task success 41.9% -> 100%

Benchmark Evidence

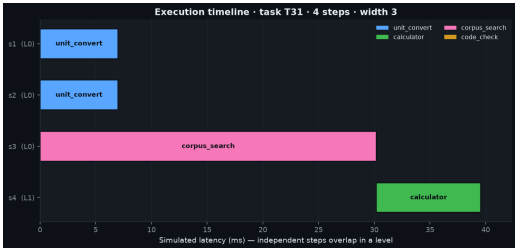
Condition	Success rate	Solved	Avg steps	Avg attempts
Without self-correction	41.9%	13/31	1.35	1.00
With self-correction	100.0%	31/31	1.35	1.58

Result Interpretation

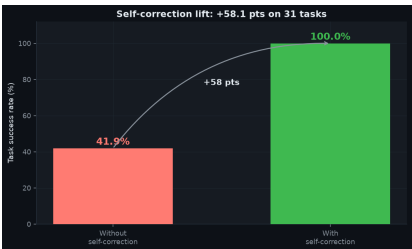
- The benchmark clearly shows that recovery logic, not just initial planning, drives system reliability.
- Guardrails reduce silent failures by stopping bad tool usage before downstream tasks compound the error.
- The measured cost-per-task makes the orchestration trade-off legible instead of qualitative.



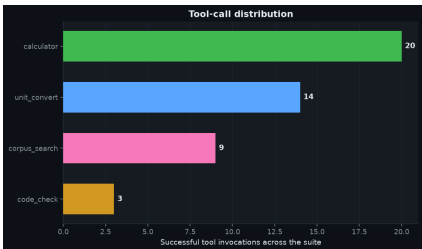
Kpi Dashboard



Execution Gantt



Success Rate



Tool Distribution

Risks And Next Steps

Operational considerations, mitigation choices, and forward path.

Risk Register

Planner brittleness Offset with typed task graphs and a repair path that can rewrite failing steps.

Tool misuse Mitigated through validators, argument checks, and execution guardrails.

State explosion Controlled by a bounded feedback loop and clear task dependency rules.

Next Steps

- Replace the deterministic policy with a stronger planner while preserving the orchestration surface.
- Add long-horizon memory and tool confidence scoring for larger workflows.
- Instrument trace-level telemetry so repair behavior can be audited like any other workflow system.

Closing Note

The project brief is intentionally written as delivered work rather than a transfer note. It documents the business problem, the engineering choices, the measured evidence, and the remaining operational path in a format suitable for portfolio review.

Sai Veda

Project brief date: February 14, 2025

Self-correction lifts task success 41.9% -> 100%